

Designing Digital Twins for Progressive Fidelity: A Layered Architecture for Airport Operations Modelling with AI-Assisted Development

Arthur Chevallier

France

`arthurprivate@laposte.net`

Abstract. Digital twins of complex cyber-physical systems are typically built as monolithic, closed-loop models that are difficult to extend and impossible to reproduce. We propose a layered fidelity architecture in which a digital twin begins as a pure event-driven simulation and progressively acquires physical, geospatial, and decisional capabilities - each phase building on the previous without architectural rewrites. To demonstrate this approach, we present Arthur International Airport (KART), a fully open-source, airport digital twin implemented using a specification-first development methodology with AI coding agent assistance. We show that structuring domain knowledge as machine-readable specification and skill files - readable by both human developers and large language model agents - reduces architectural errors and revision cycles in complex system development. The resulting system models flight operations, passenger flows, baggage handling, weather impacts, and hazardous incident cascades across seven microservices, with several layers such as LLM and a real-time geospatial interface. All artefacts, including specifications, skill files, and infrastructure code, are publicly available.

Keywords: Digital twins · Airport operations · Cyber-physical systems · AI-assisted software engineering · Specification-driven development · Simulation architecture · Progressive fidelity.

1 Introduction

- Context: airports are canonical cyber-physical systems; operational digital twins are typically monolithic, closed-loop models that are hard to extend and impossible to reproduce.
- Problem: fidelity expectations grow over a project's lifetime (logical operations → physical layout → real-world data → decision support); monolithic designs force an architectural rewrite at every step.
- **Research question.** How should digital twins of complex cyber-physical systems be architected so that fidelity can be acquired *progressively* — from pure event-driven simulation to physical, geospatial, and decisional layers — without architectural rewrites, and can specification-first development with AI coding agents make such systems tractable for a single developer?

- Sub-questions: (RQ1) what layered architecture allows progressive fidelity acquisition; (RQ2) how do machine-readable specifications and skill files affect the correctness of LLM-agent-assisted implementation; (RQ3) what does a complete end-to-end implementation reveal about costs, benefits, and failure modes.
- Contributions: (i) a layered fidelity architecture; (ii) a specification-first methodology with skill files for AI coding agents; (iii) KART, a fully open-source airport digital twin (~44k LOC Python across nine services, ~28k LOC React/TypeScript); (iv) honest lessons learned from fifty-plus sprints of agent-assisted development.
- Paper roadmap paragraph.

2 Related Work

- Digital twin foundations: origin of the concept (Grieves), the NASA/USAF formalisation (Glaessgen and Stargel), and industrial state-of-the-art surveys (Tao et al.) [2, 3, 1, 4].
- Digital twin capability and maturity models: spatially intelligent DT characteristics; smart-city deployment guidelines (from **references/** corpus).
- Airport-specific simulation and digital twins: gate/runway assignment, passenger flow modelling, delay propagation.
- AI-assisted software engineering: LLM coding agents, agentic workflows, context engineering, retrieval of project knowledge.
- Gap: no prior work combines a progressive-fidelity DT architecture with machine-readable specification artifacts designed to be consumed by both humans and LLM agents.

3 Layered Fidelity Architecture

- Layer definitions:
 - **L0 — logical event-driven simulation:** domain state machines, Kafka event bus, graph-persisted state; no geometry, no external data.
 - **L1 — physical layer:** spatial layout (terminals, gates, runways, taxiways), distances, walking and taxi times.
 - **L2 — geospatial and real-world data layer:** map-based visualisation (Mapbox/Cesium), ADS-B flight overlay, live and historical METAR weather, BTS schedule calibration, OurAirports/OpenFlights reference data.
 - **L3 — decisional layer:** LightGBM queue forecasting, what-if and Monte-Carlo scenario analysis, DCF investment appraisal, autonomous operations with approval-gated commands.
- Architectural invariants that make layers composable without rewrites: simulation-clock contract (no wall-clock reads), facts-only event bus, Neo4j as single source of truth, every state change emits an event.

- Capability ladder within each layer: visualisation → monitoring/alerting → prediction → prescription → actionability.
- Key property: each layer strictly consumes the contracts of the layer below; upgrading a layer never breaks lower ones (evidenced by the phased roadmap history landing without re-platforming).

4 Specification-First Development with AI Agent Integration

- Write-before-code discipline: event-bus schemas, graph data model, simulation rules, and one specification per service precede any implementation; the spec is authoritative and deviations require updating the spec first.
- Skill-file format: concept explanation + copy-paste patterns + gotchas; cross-cutting skills (Neo4j/Cypher, Kafka producer/consumer, simulation clock, forecasting) plus one skill per service.
- Agent entry points (`CLAUDE.md`, `AGENTS.md`, editor rule files) route the agent to the relevant spec and skill before code is written.
- Hard rules as guardrails against known LLM failure modes: no wall-clock time in business logic, no service-to-service HTTP, no silent mutations, all data synthetic.
- Feedback loop: sprint post-mortems promote discovered gotchas back into skill files — a living knowledge base shared by humans and agents.

5 The KART Implementation

- Stack: nine FastAPI domain services, Node.js/Express API gateway, React/TypeScript dashboard suite; Kafka backbone; Neo4j graph store; Prometheus/Grafana/Loki/Jaeger observability; Kubernetes manifests and CI pipeline.
- Domain depth: nine-state flight lifecycle FSM with runway queueing and five-hop delay cascades; passenger flow with security congestion and a Light-GBM model predicting queue depth 90 minutes ahead; thirty-zone baggage conveyor with dangerous-goods screening; four-state weather FSM generating ICAO-format METAR/TAF; rule-based incident cascade engine covering five hazard types.
- Scale: 420 daily movements, ~30,000 passengers and ~36,000 baggage items per simulated day; configurable clock up to 3600× real time.
- Decisional layer realised in production code: financial service (P&L, EU261 liability, carbon accounting), planning service (Monte-Carlo infrastructure scenarios, NPV/IRR investment appraisal on Eurocontrol Standard Inputs), analysis service (anomaly detection, slot-allocation MILP, autonomous mode with approval queue and per-domain command topics).
- Reproducibility: all specifications, skill files, infrastructure definitions, and data pipelines are open source; the airport itself is config-driven (a new airport is a YAML file, not code).

6 Evaluation

- Method: qualitative case study of a single-developer project across fifty-plus sprints, using the repository’s documented revision history as the evidence trail.
- Engineering-quality metrics: 760+ unit tests including event-chain contract tests and consumer idempotency tests; staged CI (lint, unit, integration, Docker build, end-to-end smoke, load tests).
- Functional demonstrations: end-to-end cascade walkthrough (weather degradation → runway capacity drop → departure queue → gate conflicts → missed connections → EU261 exposure); counterfactual replay of past decisions; Monte-Carlo investment scenario comparing baseline vs. expanded infrastructure.
- Methodology evidence: single-source-of-truth tests introduced after refactors lock constants and schemas against drift between services — the class of architectural error the skill files are designed to prevent.
- Honest scope: no controlled human study; all data is synthetic by design; quantitative claims are limited to system-level engineering metrics.

7 Discussion

- What worked: contract-first discipline; config-driven airport identity (Open/Closed principle); promoting duplicated transport code into a shared library; hexagonal refactors that made domain logic unit-testable without containers.
- Iterating on the base: a mid-project architectural audit surfaced defects characteristic of fast agent-assisted development at scale (constants diverging between copies, recommendations displayed but never executed, ML endpoints reporting success without training). All were remediated through the same spec-first process — evidence that the methodology self-corrects.
- Cautionary finding: without systematic review, such defects can pass unnoticed and even reach user-facing dashboards; audit and mutation-tested guards are a first-class part of the workflow, not an afterthought.
- Trustworthiness principle: delete or make real anything that advertises capability it lacks; predicted-vs-actual outcome measurement closes the learning loop.
- Generalisation: the layered-fidelity template and the spec/skill methodology transfer to other cyber-physical domains (ports, hospitals, warehouses).
- Threats to validity: single developer, single domain, self-reported revision-cycle claims, synthetic ground truth.

8 Conclusion

- Answers: (RQ1) a four-layer architecture with strict upward-only dependencies supports progressive fidelity without rewrites; (RQ2) machine-readable

specs and skill files measurably steer LLM agents toward architecturally consistent implementations; (RQ3) the complete implementation confirms feasibility while exposing the review practices needed to keep agent-generated systems trustworthy.

- Progressive fidelity plus specification-first agent development makes airport-scale digital twins tractable for a single developer; all artefacts are open.
- Future work: completing command-topic coverage across all domains, multi-agent reinforcement learning benchmarks using the twin as environment, and multi-airport network studies.

References

1. Glaessgen, E., Stargel, D.: The digital twin paradigm for future NASA and US air force vehicles (2012)
2. Grieves, M.: Digital twin: manufacturing excellence through virtual factory replication. White Paper (2014)
3. Grieves, M., Vickers, J.: Digital twin: mitigating unpredictable, undesirable emergent behavior in complex systems pp. 85–113 (2017)
4. Tao, F., Zhang, H., Liu, A., Nee, A.Y.C.: Digital twin in industry: state-of-the-art. *IEEE Transactions on Industrial Informatics* **15**(4), 2405–2415 (2019)